Entity resolution at scale by certified decomposition of the minimum cost multicut problem

Nov 12, 2026 · @Daniel

Personal project. CY Tech, Cycle Ingénieur, Artificial Intelligence.

1 Overview

This project builds and evaluates a method that makes the minimum cost multicut formulation of entity resolution operable: bounded memory per unit of work, incremental updates, and a reported distance to the optimum. The method is called certified decomposition in the remainder of this document.

The project consists of six steps. Firstly, we synthesize record sets whose true partition into entities is known by construction, and we complement them with public benchmarks. Secondly, we turn a record set into an instance of the minimum cost multicut problem. Thirdly, we measure where the monolithic resolution of this instance breaks as the population grows and as confusable entities become denser. Fourthly, we implement certified decomposition, which divides the instance into kernels of bounded size, reduces them by transformations that provably preserve the optimum, solves them independently, and returns a lower and an upper bound on the optimal cost together with the partition. Fifthly, we extend the method to the insertion of records one at a time. Sixthly, we compare the result to transitive closure and to the unbounded multicut, in accuracy, in computation time and in memory.

The question we ask is not whether the multicut formulation is more accurate than thresholding followed by transitive closure. It is how much of that advantage survives once the size of every optimization problem is capped, and whether the loss can be bounded by a number computed from the instance itself.

Every ingredient of the method exists in the literature. The contribution of the project is their assembly into one pipeline with a certificate, and the measurement of its behaviour on entity resolution workloads.

2 Problem statement

Let V be a finite set of records and let E be a set of candidate pairs of records. For every pair uv in E we are given a cost c_uv in R, positive when the evidence favours the two records designating the same entity and negative when it favours the opposite. We seek a partition of V minimizing the sum of the costs of the pairs separated by the partition. Written on the edges, with x_uv = 1 if uv is cut:

```
\min_{x \in \{0,1\}^{E}} \; \sum_{uv \in E} c_{uv}\, x_{uv}
\qquad \text{subject to} \qquad
\forall C \in \mathrm{cycles}(G),\ \forall e \in C:\quad x_e \le \sum_{f \in
C \setminus \{e\}} x_f
```

The cycle inequalities state that no cycle may be cut exactly once, which makes the feasible set exactly the set of partitions of V. Transitivity is therefore a constraint of the model and not a post-processing step. We write OPT(G) for the optimal value and cost(x) for the value of a feasible x.

A second form of the objective is used throughout. An edge is said to be violated by a partition when it is positive and cut, or negative and joined. The cost of a partition equals a constant plus the total absolute cost of its violated edges:

```
\mathrm{cost}(x) \;=\; \underbrace{\sum_{e \in E} \min(0, c_e)}_{L_0} \;+\;
\sum_{e\,:\,c_e > 0} c_e\, x_e \;+\; \sum_{e\,:\,c_e < 0} |c_e|\,(1 - x_e)
```

The constant L_0 is the cost that would be reached if every edge could be satisfied at once. It is the weakest lower bound on OPT(G), and the starting point of the stronger ones of section 5.

The problem is NP-hard. On a sparse graph the cycle inequalities that matter are those of the chordless cycles, and their number is exponential in general. Only on a complete graph do they reduce to the triangle inequalities, whose number is cubic in the number of vertices.

3 Limits of the monolithic approach

Four obstacles separate the formulation of section 2 from a system that can be operated. None of them is the NP-hardness of the problem as such, since a greedy solver handles very large sparse graphs on one machine. Each of them is stated here as a claim that the project must first measure, in stage 1, before attempting to remove it.

The giant component. The usual argument for scalability is that the candidate graph decomposes into connected components that are solved separately. Component sizes are however heavy-tailed. Shared addresses and frequent names link many records into one component, which then holds a large share of the population and all of the difficult cases. The decomposition gives no help where help is needed.

The missing repulsive evidence. A pair that is absent from E has cost zero. Blocking retains pairs that resemble each other, so the candidate graph contains mostly positive edges. Consider a chain of three records a, b, c in which ab and bc are positive and ac was never scored. The multicut optimum joins the three records, exactly as transitive closure does. The advantage of the formulation exists only if the negative evidence between a and c is present in the instance.

The absence of an incremental mode. A new record may in principle change the optimal partition far from its neighbours. A full resolution after every insertion is not affordable, and a heuristic re-run may relabel clusters that received no new evidence, which breaks the identifiers on which downstream systems depend. Transitive closure has neither problem: union-find absorbs an insertion in near-constant time and never splits a cluster.

The absence of a certificate. The solver that scales, greedy additive edge contraction, has no approximation guarantee. The solver that certifies, integer programming, does not terminate on large components. At scale one therefore obtains a partition without any statement on its distance to the optimum.

4 Proposed solution

Certified decomposition replaces one large optimization problem by many problems of at most B records, called kernels, and accounts exactly for what the replacement may cost. The integer B is the single parameter of the method. The pipeline is a straight run of seven steps.

1. **Blocking and scoring.** A candidate set E is produced and every candidate pair receives a cost. This step is unchanged from the original project.
2. **Kernel formation.** The positive edges are processed in decreasing order of cost and their endpoints are merged, unless the merge would create a group of more than B records. The resulting groups are the kernels. The positive edges left between two kernels are said to be abandoned, and their total cost is denoted Δ.

3. **Completion.** Inside every kernel, all pairs that blocking did not propose are scored. Every kernel becomes a complete graph, which restores the negative evidence of section 3.
4. **Reduction.** Inside every kernel, edges that satisfy a persistency criterion are contracted. The criterion guarantees that an optimal solution joins their endpoints, so the optimum is unchanged and the kernel shrinks.
5. **Resolution.** Every reduced kernel is solved by greedy additive edge contraction followed by a local search, and by integer programming when it is small enough.
6. **Bounding.** Every kernel receives a lower bound on its optimal cost: the integer programming optimum when available, a cycle packing bound otherwise.
7. **Assembly.** The partition of V is the union of the kernel partitions. It is returned with an upper bound U, a lower bound L and the gap U − L.

Two limiting cases explain the design. With B infinite, the kernels are the connected components of the positive subgraph and $\Delta = 0$: by proposition 1 below, nothing is lost. With B infinite and steps 3 to 6 disabled, the output is exactly thresholding at $p = 1/2$ followed by transitive closure. The method is therefore an additional stage placed after the procedure that is standard in practice, and it degrades to that procedure when it is switched off.

The instance that the certificate refers to is the completed one: the candidate pairs of step 1 together with the pairs added in step 3. Pairs of records in different kernels that blocking did not propose are never scored and keep cost zero. This is a limitation of the certificate, and it is stated in the report.

5 Mathematical foundations

Three propositions justify the pipeline: the first makes kernel formation exact when B is infinite, the second makes the reduction exact, and the third prices the cap B. All three are proved by exchange arguments short enough to be reproduced in the report and checked by brute force in the tests.

Throughout, $G = (V, E)$ is the instance with costs c, $\delta(U)$ is the set of edges with exactly one endpoint in a subset U of V, and G^+ is the subgraph of G that keeps only the edges of positive cost.

5.1 Multicut refines transitive closure

Proposition 1. There exists an optimal partition in which every cluster is contained in one connected component of G+.

Proof. Take an optimal partition and a cluster S that meets several components of G+. Replace S by its intersections with these components. The only edges whose status changes are those between two different components of G+ inside S. They go from joined to cut, and none of them is positive, so the cost does not increase. Repeating the operation for every cluster gives the claim.

The consequence is that the multicut optimum can only split the clusters of transitive closure at p = 1/2, never merge them. The connected components computation that industry already runs is an exact first stage of the multicut resolution.

5.2 Persistency of strongly supported joins

Proposition 2. Let uv be an edge with c_uv > 0. If there exists a subset U of V that contains u and not v such that

```
c_{uv} \;\ge\; \sum_{e \in \delta(U) \setminus \{uv\}} |c_e| ,
```

then there exists an optimal partition in which u and v belong to the same cluster.

Proof. Take an optimal partition in which u and v are separated, with clusters S_u and S_v. Let A be the part of S_u inside U and let B be the part of S_v outside U. Form a new partition by removing A from S_u, removing B from S_v, and adding the union of A and B as a cluster. Three families of edges change status: those between A and the rest of S_u, those between B and the rest of S_v, and those between A and B. All three are contained in δ(U). The edge uv goes from cut to joined, which lowers the cost by c_uv. Every other edge of δ(U) raises the cost by at most its absolute cost. Under the hypothesis the new partition is therefore no more expensive, and it joins u and v.

The project uses the cheapest instance of the criterion, U = {u}: the edge uv is contracted when its cost is at least the sum of the absolute costs of all other edges incident to u, or the same at v. The test costs a number of operations proportional to the degree of the vertex.

Contraction. Contracting uv merges u and v into one vertex and replaces the two edges wu and wv, for every other vertex w, by one edge of cost c_wu + c_wv. Partitions of the contracted graph correspond exactly to partitions of G that join u and v, with equal cost. Under proposition 2 the optimal value is therefore unchanged.

Order of application. Proposition 2 asserts the existence of one optimal partition per edge, not of one partition for all edges at once. The reduction must therefore contract one edge, update the costs, and test again. Two edges that share no vertex may be contracted in the same pass: the contraction of one never increases the sum of absolute costs around the endpoints of the other and leaves its cost unchanged.

5.3 The price of bounded kernels

Proposition 3. Let R_1, ..., R_m be any division of V into regions, and let E_× be the set of edges between two distinct regions. Then

```
\sum_{i=1}^{m} \mathrm{OPT}(G[R_i]) + \sum_{e \in E_\times} \min(0, c_e)
\;\le\; \mathrm{OPT}(G) \;\le\;
\sum_{i=1}^{m} \mathrm{OPT}(G[R_i]) + \sum_{e \in E_\times} c_e .
```

Proof. For the right inequality, the union of optimal partitions of the regions is a partition of V in which every edge of E_× is cut, and its cost is the right-hand side. For the left inequality, restrict an optimal partition of G to each region. The restriction is feasible for the region, so the edges inside R_i cost at least OPT(G[R_i]). The edges of E_× cost at least the sum of their negative costs, which is reached when all negative ones are cut and all positive ones are joined.

The difference between the two sides is the quantity

```
\Delta \;=\; \sum_{e \in E_\times} \max(0, c_e) ,
```

the total cost of the abandoned positive edges. Solving the regions independently therefore costs at most Δ more than the optimum. When the regions are the components of G+, Δ = 0 and proposition 1 is recovered.

5.4 Lower bounds inside a kernel

Proposition 3 requires the optimal value of every kernel, which is available only when the integer program terminates. Otherwise a lower bound is used in its place, and the left inequality remains valid. Three bounds are implemented, from weakest to strongest.

Trivial bound. L_0, the sum of the negative costs, defined in section 2.

Cycle packing bound. In a complete kernel, call a triangle conflicted when exactly one of its three edges is negative. No partition can satisfy its three edges: if both positive edges are joined, the negative one is joined as well. Every partition therefore violates at least one

edge of every conflicted triangle. Give every edge a residual weight, initially its absolute cost. For each conflicted triangle whose three residual weights are positive, let m be the smallest of the three, subtract m from each and add m to a running total P. Then

```
L_{\mathrm{pack}} \;=\; L_0 + P \;\le\; \mathrm{OPT} .
```

The proof uses the second form of the objective. The cost above L_0 is the total absolute cost of the violated edges. Each triangle of the packing holds a share m of the weight of each of its edges, the shares held on one edge never exceed its absolute cost, and each triangle has at least one violated edge. The violated edges therefore carry at least P.

Integer programming bound. The optimal value of the integer program when it terminates, and its best proven bound when it reaches the time limit. Both are valid because a program with only a subset of the cycle inequalities is a relaxation of the problem.

5.5 The certificate

Let $\hat{x}$ be the partition returned by the pipeline, let U_K be the cost of its restriction to kernel K, and let L_K be the best lower bound available for K. The pipeline reports

```
U = \sum_{K} U_K + \sum_{e \in E_\times} c_e , \qquad
L = \sum_{K} L_K + \sum_{e \in E_\times} \min(0, c_e) , \qquad
U - L = \underbrace{\sum_{K} (U_K - L_K)}_{\text{solver gap}} +
\underbrace{\Delta}_{\text{decomposition gap}} .
```

By propositions 2 and 3, $L \leqslant \mathrm{OPT}(G) \leqslant U$. The gap separates what is lost by the heuristic solver from what is lost by the cap B, which tells which of the two to improve. The relative gap is defined as

```
\gamma \;=\; \frac{U - L}{U - L_0} \;\in\; [0, 1] ,
```

with $\gamma = 0$ by convention when $U = L_0$. It is the fraction of the violation cost of the returned partition that has not been proved unavoidable: $\gamma = 0$ certifies optimality, and $\gamma = 1$ means that nothing has been certified.

6 Data

The synthetic generator remains the primary source, because it is the only one that lets the difficulty be controlled. Public benchmarks are added so that no conclusion rests on data that the author designed.

6.1 Synthetic records

We synthesize a population of n entities, each carrying an attribute record: family name, given name, date of birth, address, postal code and a partial identifier. For every entity we sample between one and k observations distributed over two systems A and B, through a corruption model with a scalar severity θ that governs three levels of perturbation.

- **Character level.** Insertion, deletion, substitution and transposition, at a rate increasing with θ.
- **Token level.** Inversion of given name and family name, abbreviation to initials, omission of a middle name, particle variants.
- **Field level.** Missing values, alternative date and postal code formats, truncation of the identifier.

A second mechanism controls the density ρ of confusable entities, that is, pairs of distinct entities placed close in attribute space. Two constructions are used: members of one household, who share family name and address and differ in given name and date of birth, and subsidiaries of one group, which share a legal root name and a registered address and differ in a legal suffix. The density ρ is the parameter that creates the giant component, and it is swept up to the point where a single component of G+ holds a substantial share of the records.

Three populations are generated per configuration, with distinct seeds: one to train the pairwise classifier, one to calibrate it, one to evaluate. The true partition is used only in evaluation. Population sizes are n = 10^4, 10^5 and 10^6.

For the incremental experiments, the evaluation records are also written as a stream in a random order fixed by the seed.

6.2 Public benchmarks

- **Febrl 4 (required).** Two files of 5,000 records each with known links, distributed with the Python package recordlinkage. It is small and serves as a sanity check against published results.

- **MusicBrainz 20K (recommended).** A multi-source benchmark of the Leipzig database group with ground-truth clusters of up to five records, which is closer to the multi-record setting than a two-file linkage.
- **North Carolina voter registration (optional).** Real person records across snapshots, with the voter identifier as ground truth. It is the largest of the three and requires preprocessing.

The availability and the licence of each benchmark are to be checked before use. On benchmarks, the classifier is trained on a labelled subset of the benchmark itself, disjoint from the evaluation entities.

7 Construction of the instance

7.1 Blocking

The set of all pairs is reduced to a candidate set E by the union of several blocking keys, for instance sorted neighbourhood over (family name, date of birth) and over (postal code, given name), and an inverted index over character 3-grams of the full name. We report the reduction ratio and the pair completeness. We also report a third quantity that the original project did not need: the share of candidate pairs that are true non-matches. It measures how much negative evidence blocking leaves in the instance before completion.

7.2 Pairwise evidence

For every scored pair we compute a vector of similarity features: Jaro-Winkler and normalized Levenshtein similarities on the name fields, token Jaccard on the address, calendar distance on the date of birth, agreement indicators on the identifier fragment and the postal code, and one missingness indicator per field. The same function scores the candidate pairs of blocking and the pairs added by completion, so that all costs are comparable.

7.3 From probability to cost

We estimate p_{uv}, the probability that u and v designate the same entity given the features, by logistic regression, and we set

$$c_{uv} \;=\; \log \frac{\tilde p_{uv}}{1 - \tilde p_{uv}} , \qquad \tilde p_{uv} = \min\bigl(1 - \varepsilon,\ \max(\varepsilon,\ p_{uv})\bigr) .$$

Under an independence assumption on the pairwise evidence, minimizing the multicut objective with these costs is maximum a posteriori estimation of the partition. The sign of the cost changes at p = 1/2, so the decision threshold of the classifier is the zero of the cost and no further threshold is introduced.

The clipping constant ε bounds every cost in absolute value by log((1 − ε)/ε), about 9.2 for ε = 10^-4. Without it, a single overconfident pair can outweigh a whole neighbourhood and can trigger the persistency criterion of proposition 2 on its own.

7.4 Calibration and the training distribution

The costs inherit any miscalibration of p directly. A calibration set is held out, the reliability diagram is inspected, and isotonic regression is applied if required.

Completion changes the distribution of the pairs that the classifier scores: the pairs it adds are, by construction, pairs that blocking rejected, and most of them are non-matches that share one attribute through a chain. The classifier must therefore be trained and calibrated on a sample that includes completion pairs, obtained by running steps 1 to 3 of the pipeline on the training population. Calibration is reported separately on blocking pairs and on completion pairs.

8 Algorithms of the batch pipeline

Every algorithm below is deterministic: ties are broken by record index, and no step depends on the number of workers. The pseudocode fixes the behaviour that the tests of section 11 check.

8.1 Kernel formation

Kernel formation is Kruskal's algorithm on the positive edges, in decreasing order of cost, with a cap on the size of the merged sets.

```
sort the positive edges by (cost descending, u ascending, v ascending)
uf    <- union-find over V, tracking set sizes
delta <- 0
for (u, v, c) in sorted edges:
    a, b <- uf.find(u), uf.find(v)
    if a == b: continue
    if size[a] + size[b] <= B: uf.union(a, b)
    else: delta <- delta + c          # abandoned edge
kernels <- the sets of uf
```

An edge abandoned at one point never becomes internal to a kernel later, since sets only grow and the union of its two sets already exceeded B. The accumulated value is therefore exactly Δ. The cost is O(|E| log |E|). The strongest edges are merged first, so the abandoned edges are the weakest ones available at the moment the cap is reached. Better splitting rules exist, based on minimum cuts, and are out of scope: the cap rule is sufficient because Δ reports what it costs.

8.2 Completion

Every kernel K of s records, with s at least 3, is stored as a dense symmetric s by s matrix C with zero diagonal. The entries of the candidate pairs are copied from step 1, and the remaining pairs are scored in one batch. The number of scored pairs over all kernels is at most B|V|/2 and far lower in practice, since most kernels are small. Kernels of one or two records need no completion.

8.3 Reduction by persistency

At a vertex u, only the largest positive edge can satisfy the criterion of proposition 2, so one test per vertex suffices.

```
active  <- all vertices of the kernel
members <- {i: {i}}
repeat:
    changed <- false
    for u in active (ascending):
        v <- argmax over active w != u of C[u, w]
        a <- sum over active w != u of |C[u, w]|
        if C[u, v] > 0 and C[u, v] >= a - C[u, v]:
            contract(u, v)
            changed <- true
until not changed

contract(u, v):                       # v absorbs u
    C[v, :] <- C[v, :] + C[u, :]
    C[:, v] <- C[:, v] + C[:, u]
    C[v, v] <- 0
    remove u from active; members[v] <- members[v] | members[u]
```

The loop contracts one edge at a time and recomputes on the updated matrix, as section 5.2 requires. We report the reduction rate, that is, the share of vertices removed, per kernel size.

8.4 Greedy additive edge contraction

```
loop:
    (u, v) <- argmax over active pairs of C[u, v]
    if C[u, v] <= 0: break
    contract(u, v)
clusters <- members of the remaining active vertices
```

On a dense matrix this simple form costs O(s^3) per kernel, which is acceptable for B up to a few hundred. A priority queue brings it to O(s^2 log s) and is an optimization to be made only if profiling asks for it.

8.5 Local search

Starting from the greedy clusters, a vertex i of the reduced kernel is moved from its cluster A to another cluster T, or to a new empty cluster, when the move lowers the cost. The change in cost is

```
\mathrm{gain}(i, A \to T) \;=\; \sum_{w \in A \setminus \{i\}} C_{iw} \;-\;
\sum_{w \in T} C_{iw} ,
```

and the move is applied when the gain is below −10^-9. Sweeps over the vertices are repeated until no move applies. Every move strictly lowers the cost, so the search terminates. Kernighan-Lin with joins is the stronger variant and is optional.

8.6 Exact resolution

Reduced kernels of at most s_max vertices are also solved by integer programming, with one binary variable per pair and the triangle inequalities x_ij ⩽ x_ik + x_kj. Up to about 40 vertices all triangle inequalities are stated at once. Beyond that, they are separated lazily: solve, list the violated triangles of the solution, add them, solve again. A time limit per kernel is enforced. On time-out the best proven bound of the solver is kept as a lower bound and the greedy solution is kept as the partition. The open solver HiGHS, reachable through scipy.optimize.milp, is sufficient at this size.

8.7 Bounds and assembly

For every kernel the pipeline stores U_K, the cost of the retained partition, and L_K, the largest of the trivial bound, the cycle packing bound of section 5.4 and the integer programming bound. The global values U, L, Δ and γ follow from section 5.5. As a check, U

is also recomputed directly from the final labels over all edges of the completed instance, and the two values must agree to within floating-point tolerance.

8.8 Parallel execution

Steps 3 to 7 treat kernels independently. Kernels are sorted by decreasing size and distributed over a pool of worker processes. Peak memory per worker is one matrix of B^2 costs. The results must be identical for any number of workers.

9 Incremental mode

The incremental mode inserts one record at a time and touches only the kernels that the record is positively linked to. Its work per insertion is bounded by a function of B and does not depend on the size of the population, apart from the blocking lookup. Deletions and modifications of records are out of scope.

9.1 State

The state consists of the blocking index, the kernel and the entity identifier of every record, the cost matrix and the bounds U_K and L_K of every kernel, and the running values of Δ, U and L.

9.2 Insertion of a record r

1. **Candidates.** Query the blocking index, score the candidate pairs, and let P be the positive edges from r to existing records.
2. **Touched kernels.** Let T be the set of kernels that contain an endpoint of P. If T is empty, r forms a new kernel and receives a new entity identifier, and the insertion ends.
3. **Region.** Sort the kernels of T by decreasing total positive cost to r. Add them to a region R, initialised to {r}, as long as the size of R stays at most B. The positive edges from r to the kernels left out are abandoned and their costs are added to Δ.
4. **Completion.** Score r against every record of R. If several kernels were merged, also score the pairs between them.
5. **Resolution.** Apply the reduction, the greedy contraction and the local search of section 8 to R.

6. **Safeguard.** Compute the cost of the trivial update, in which the previous clusters are kept and r is a singleton. Keep whichever of the two partitions is cheaper. The cost of the region therefore never exceeds that of the trivial update, although the solver is heuristic.
7. **Identifiers.** Relabel the clusters of R by maximum overlap, as described below.
8. **Certificate.** Recompute the bounds of R and update U, L and γ.

9.3 Stability of identifiers

For every pair made of a new cluster and a previous entity identifier, count the records they share. Process the pairs in decreasing order of this count, ties broken by the smaller identifier. A new cluster receives the identifier of the pair if neither has been assigned yet. Clusters left without an identifier receive a fresh one. Records outside R keep their identifier by construction, which is the property that a full re-run cannot offer.

9.4 Reconciliation with the batch pipeline

The kernels obtained from a stream depend on the order of arrival, whereas the batch pipeline processes all positive edges by decreasing cost. The two results therefore differ, and the difference is called drift. After replaying the whole stream, the batch pipeline is run on the same records and three quantities are reported: the variation of information between the two partitions, the difference of their costs U, and the difference of their decomposition gaps Δ. In operation, a periodic batch run would reset the drift, at the price of identifier changes that the maximum overlap rule keeps to a minimum.

10 Software architecture

The package is organised so that every step of section 4 is one module with one public function, testable in isolation on arrays. The name ermc is a placeholder.

Module	Responsibility	Public function
ermc/synth.py	Population, corruption model, confusable entities, stream order	generate(n, theta, rho, seed)
ermc/blocking.py	Blocking keys, inverted index, candidate pairs, index lookup for one record	candidates(records), lookup(record)

Module	Responsibility	Public function
ermc/features.py	Similarity features of a batch of pairs	featurize(records, pairs)
ermc/model.py	Logistic regression, calibration, clipped logit costs	fit(...), costs(features)
ermc/kernels.py	Capped union of section 8.1, returns kernels and Δ	form_kernels(edges, B)
ermc/completion.py	Dense cost matrix of a kernel	complete(kernel, edges, scorer)
ermc/persistency.py	Reduction of section 8.3	reduce(C)
ermc/solvers/gaec.py	Greedy additive edge contraction	solve(C)
ermc/solvers/moves.py	Local search of section 8.5	improve(C, labels)
ermc/solvers/ilp.py	Integer program with triangle inequalities	solve(C, time_limit)
ermc/solvers/brute.py	Enumeration of all partitions, for tests only	solve(C)
ermc/bounds.py	Trivial and cycle packing bounds	lower_bound(C)
ermc/pipeline.py	Steps 1 to 7, parallel over kernels, certificate	resolve(records, B)
ermc/incremental.py	State and insertion of section 9	Resolver.insert(record)
ermc/baselines.py	Transitive closure, thresholding without closure, average linkage	one function each
ermc/metrics.py	Variation of information, pairwise scores, cluster statistics	vi(a, b), pairwise(a, b)
ermc/experiments/	One script per figure, reading one configuration file	run(config)

10.1 Data structures

Records are rows of a table with an integer index from 0 to |V| − 1. Edges are three aligned arrays (u, v, c) with u < v, in 32-bit integers and 32-bit floats. A partition is one integer array of labels of length |V|. A kernel is an array of record indices and a dense matrix of costs. Solvers take a matrix and return labels: they never see records, which keeps them independent of the application.

10.2 Tooling

Python 3.11 or later, with numpy and scipy for arrays, union-find and the integer program, scikit-learn for the classifier and the calibration, rapidfuzz for string similarities, polars or pandas for tables, pytest and hypothesis for tests, and the standard multiprocessing pool for parallel kernels. The loops of sections 8.3 to 8.5 are written first in plain numpy. They are compiled with numba only if profiling shows that they dominate, and the compiled version must pass the same tests.

10.3 Reproducibility

Every experiment is defined by a configuration file that fixes the seed, n, θ, ρ, B and the solver options. One command regenerates every figure from scratch. Every run writes a results file in JSON that contains the configuration, the metrics of section 12, the certificate (U, L, Δ, γ), the wall-clock time per step, the peak memory, and the version of the code. Figures are produced only from these files, never from values held in memory.

11 Tests

The tests turn every mathematical statement of section 5 into an executable check against a brute-force solver. The brute-force solver enumerates all partitions of a set, which is feasible up to 8 vertices (4,140 partitions) and tolerable at 9 (21,147).

Random instances are complete graphs with integer costs drawn in [−5, 5], so that comparisons of costs are exact and no test depends on a floating-point tolerance. Property tests are written with hypothesis and run on at least 500 instances each.

Id	Subject	Check	Expected
T1	Brute force	Hand instance of 4 vertices solved on paper	Same partition and cost

Id	Subject	Check	Expected
T2	Chain counterexample	Records a, b, c with c_ab = 2, c_bc = 1, c_ac = −3	Transitive closure returns {a, b, c}. Multicut returns {a, b}, {c} with cost −2
T3	Chain without negative evidence	Same instance with the pair ac removed	Multicut returns {a, b, c} with cost 0, as transitive closure does
T4	Proposition 1	Random instances, optimum of G against the sum of the optima of the components of G+	Equal
T5	Proposition 2	Random instances, optimum before and after reduce()	Equal
T6	Proposition 2, hand case	Vertex u with incident costs 5, −2, 1	The edge of cost 5 is contracted
T7	Contraction	Cost of a partition of the contracted graph against the cost of the corresponding partition of G	Equal
T8	Proposition 3	Random instances and random regions	Both inequalities hold, and their difference equals Δ
T9	Kernel formation	Sizes, and Δ recomputed from the kernels	Every size at most B. Δ equals the sum of positive edges between kernels
T10	Kernel formation, no cap	B at least |V|	Kernels equal the connected components of G+ given by scipy, and Δ = 0
T11	Limiting case	Pipeline with B infinite and steps 3 to 6 disabled	Same partition as the transitive closure baseline
T12	Order of bounds	Random instances	L_0 ⩽ L_pack ⩽ OPT ⩽ cost after local search ⩽ cost after greedy contraction

Id	Subject	Check	Expected
T13	Integer program	Random instances up to 8 vertices	Cost equal to brute force, and no triangle inequality violated
T14	Local search	Random instances and random starting labels	Cost never increases, and the search terminates
T15	Certificate	Pipeline on small synthetic data	L ⩽ U, γ in [0, 1], U equal to the cost recomputed from labels
T16	Certificate, exact case	All kernels solved by integer programming and Δ = 0	γ = 0
T17	Determinism	Two runs with one seed, and runs with 1 and 4 workers	Identical labels and certificate
T18	Variation of information	Identity, symmetry, relabelling, and {1, 2}, {3, 4} against {1, 2, 3, 4}	0, equal, unchanged, and ln 2
T19	Costs	Any feature vector	Absolute cost at most log((1 − ε)/ε)
T20	Incremental, invariants	After every insertion of a short stream	Kernel sizes at most B. Records outside the region keep their identifier. Running U equals recomputed U
T21	Incremental, safeguard	After every insertion	Cost of the region at most the cost of the trivial update
T22	End to end, no noise	Generator with θ = 0 and ρ = 0	Every method recovers the true partition

Tests T2 and T3 together are the unit-test form of the argument of section 3: the same three records give two different answers depending only on whether the negative pair is present in the instance.

12 Metrics and experiments

12.1 Baselines

- **Thresholding and transitive closure** at p > 1/2, the standard industrial procedure and the reference point of the study.
- **Thresholding without closure**, which leaves the relation intransitive, as a lower reference.
- **Hierarchical agglomerative clustering** with average linkage on 1 − p, inside the components of G+.
- **Multicut without completion**: greedy contraction and local search on the candidate graph, per component of G+. This is the method of the original project.
- **Reference multicut**: certified decomposition with the largest cap B_ref that fits in memory, for instance 2,000.
- **Splink (optional)**, an open-source record linkage library that clusters by connected components, as an external point of comparison on the public benchmarks.

12.2 Accuracy

The primary metric is the variation of information between the true partition T and the predicted partition P. With n_ij the number of records shared by cluster i of T and cluster j of P, and n_i, n_j the cluster sizes:

```
\mathrm{VI}(T, P) \;=\; \underbrace{-\sum_{i,j} \frac{n_{ij}}{n} \log \frac{n_{ij}}{n_j}}_{H(T \mid P)\ \text{: false merges}} \;\; \underbrace{-\sum_{i,j} \frac{n_{ij}}{n} \log \frac{n_{ij}}{n_i}}_{H(P \mid T)\ \text{: false splits}}
```

The two terms are reported separately, because transitive closure fails by merging and a cap that is too small fails by splitting. Pairwise precision, recall and F1 are secondary: one spurious merge of two clusters of size m creates m^2 true positive pairs for a single wrong decision, so pairwise scores partly hide the failure under study.

The central derived quantity is the retention of the advantage at cap B:

```
\mathrm{ret}(B) \;=\; \frac{\mathrm{VI}_{\mathrm{closure}} - \mathrm{VI}_{B}}{\mathrm{VI}_{\mathrm{closure}} - \mathrm{VI}_{B_{\mathrm{ref}}}}
```

12.3 Structure, optimality and cost of computation

Metric	Definition	Reported in
Largest component share	Records in the largest component of G+, divided by |V|	E1
Negative evidence share	Negative edges divided by all edges, before and after completion	E2
Kernel size distribution	Histogram of kernel sizes at cap B	E3
Reduction rate	Vertices removed by persistency, divided by kernel size	E3
Decomposition gap	Δ, and Δ divided by U − L_0	E3
Solver gap	Sum over kernels of U_K − L_K	E3
Relative gap	γ of section 5.5	E3
Wall-clock time	Per step of the pipeline, and total	E1, E3, E4
Peak memory	Resident memory of the process, and of the largest worker	E1, E4
Throughput	Records resolved per second in batch	E4
Speed-up	Time with 1 worker divided by time with w workers	E4
Insertion latency	Median and 99th percentile of the time of one insertion	E5
Identifier churn	Existing records whose identifier changes, per insertion	E5
Drift	Section 9.4	E5

12.4 Hypotheses

Each hypothesis is stated with the observation that would refute it. A refutation is a reportable result and is reported as such.

Id	Hypothesis	Refuted if
H1	The largest component share grows with ρ and with n, and the time and memory of the reference multicut are dominated by that component	The largest component stays small over the whole plane. The bottleneck then does not exist on this data and per-component resolution suffices
H2	Without completion, multicut is within noise of transitive closure in variation of information. With completion it is lower, by a margin that grows with ρ	Completion makes no measurable difference
H3	There is a cap of a few hundred records at which ret(B) is at least 0.95, and Δ decreases as B grows	Retention of 0.95 requires a cap comparable to the size of the giant component
H4	At fixed B, the 99th percentile of insertion latency does not grow with n, and the drift is small compared with the advantage over transitive closure	Latency grows with n, or the drift cancels the advantage

12.5 Experiments and figures

All experiments use five seeds and report the mean and the standard deviation. The machine, its number of cores and its memory are stated in the report.

- **E1, bottleneck.** Grid over θ in {0.05, 0.1, 0.2, 0.3} and ρ in {0, 0.05, 0.1, 0.2, 0.4}, for n = 10^4, 10^5, 10^6. Figure 1: largest component share against ρ, one curve per n. Supports H1.
- **E2, negative evidence.** Same grid at n = 10^5. Figure 2: variation of information against ρ for transitive closure, average linkage, multicut without completion and certified decomposition, with the merge and split terms shown separately. Supports H2.
- **E3, the cap.** B in {25, 50, 100, 200, 400, 800} at the two highest values of ρ. Figure 3: ret(B), Δ and γ against B. Figure 4: kernel size distribution and reduction rate. Supports H3.
- **E4, scaling.** Time, memory and throughput against n at the cap chosen in E3, with 1, 2, 4 and 8 workers. Figure 5: time against n on logarithmic axes, one curve per method, and speed-up against the number of workers.
- **E5, stream.** Replay of the stream at n = 10^5 and 10^6. Figure 6: latency percentiles against B and against n, with churn and drift reported in a table. Supports H4.

- **E6, benchmarks.** Accuracy and certificate of every method on Febrl 4 and on the second benchmark, in one table.

13 Plan, risks and review

13.1 Stages

The project is feasible for one person in roughly 55 to 75 hours, assuming that working code is produced with the assistance of a coding model and that the time is spent on design, tests and analysis. Stages 0 to 2 with Febrl 4 form the minimum project that supports the thesis. Stages 3 and 4 are the extension.

Stage	Content	Completed when	Hours
0	Generator, blocking, features, classifier, costs, transitive closure, variation of information, greedy contraction on the candidate graph	T1 to T3, T18, T19 and T22 pass, and one sweep over θ is plotted	12 to 15
1	Bottleneck measurement	Experiment E1 is run and H1 is decided	4 to 6
2	Kernel formation, completion, persistency, local search, integer program, bounds, certificate, parallel pool	T4 to T17 pass, and experiments E2 and E3 are run	15 to 20
3	Incremental mode	T20 and T21 pass, and experiment E5 is run	8 to 12
4	Scaling and benchmarks	Experiments E4 and E6 are run	8 to 12
5	Report of eight to twelve pages, README	One command reproduces every figure	6 to 8

13.2 Risks

- **No giant component appears.** If H1 is refuted on the synthetic data, the households and groups of section 6.1 are too small or too rare. The mitigation is to increase their size and ρ, and to check the public data. If the component remains small there too, the result is that per-component resolution suffices, and it is reported.

- **Persistency fires rarely.** In a complete kernel, a vertex has many negative edges, so the sum on the right of the criterion is large. The reduction is expected to act mostly on small and clean kernels. Its rate is a measured quantity and not a prerequisite: the pipeline is correct with a reduction rate of zero.
- **Completion pairs are miscalibrated.** They come from a distribution that the classifier did not see unless section 7.4 is followed. A miscalibrated negative cost produces false splits, which the split term of the variation of information reveals.
- **Completion dominates the run time.** Its cost grows with the sum of the squared kernel sizes. It is measured per step, and B is the lever.
- **The lower bound is weak.** The cycle packing bound may leave γ large although the partition is good. γ is an upper bound on the loss, not an estimate of it. The mitigation is to solve more kernels exactly, and to report the solver gap and the decomposition gap separately.
- **The cap abandons strong edges.** If two large groups meet through an edge of high cost, Δ grows sharply. This is visible in the certificate. A splitting rule based on minimum cuts is the natural extension and is out of scope.
- **Conclusions limited to synthetic data.** At least one public benchmark is required, and the report states which conclusions were observed on which data.
- **Scope creep.** Learned matchers, lifted multicuts, distributed engines and deletions are out of scope. The pairwise classifier is kept simple on purpose, because it isolates the effect of the partitioning step.

13.3 What the project can and cannot claim

The project can establish, at up to 10^6 records on one machine, bounded memory per worker, insertion latency independent of the population size, and a certified gap. It cannot establish commercial viability, which depends on data, volumes and constraints that a personal project does not observe. The report uses the first formulation.

13.4 Material to bring back for review

The review will cover both the implementation and the results, and requires the following.

1. The repository, with the exact commit of the results.
2. The output of the test suite, with the list of tests of section 11 that pass, fail or were not implemented.
3. The results files of experiments E1 to E6, and the six figures.
4. The description of the machine, and the wall-clock time of the full reproduction.

5. The reliability diagrams of the classifier, on blocking pairs and on completion pairs.
6. A list of every deviation from this document, with its reason.
7. The draft of the report, if it exists.

The review will assess, in this order: the correctness of the solvers and of the certificate against the tests, the validity of the experimental protocol (separation of training, calibration and evaluation data, seeds, absence of leakage of the true partition), whether each conclusion on H1 to H4 follows from the figures, and the reproducibility and clarity of the code.

References

The first list is carried over from the original project document. The second list was added for this version and is cited from memory: titles, venues and years are to be verified before they enter the report.

Carried over

1. Fellegi, I. P. & Sunter, A. B. (1969). A theory for record linkage. Journal of the American Statistical Association 64(328).
2. Bansal, N., Blum, A. & Chawla, S. (2004). Correlation clustering. Machine Learning 56(1–3).
3. Demaine, E. D., Emanuel, D., Fiat, A. & Immorlica, N. (2006). Correlation clustering in general weighted graphs. Theoretical Computer Science 361(2–3).
4. Chopra, S. & Rao, M. R. (1993). The partition problem. Mathematical Programming 59.
5. Kappes, J. H., Speth, M., Andres, B., Reinelt, G. & Schnörr, C. (2011). Globally optimal image partitioning by multicuts. EMMCVPR 2011.
6. Keuper, M., Levinkov, E., Bonneel, N., Lavoué, G., Brox, T. & Andres, B. (2015). Efficient decomposition of image and mesh graphs by lifted multicuts. ICCV 2015. Introduces greedy additive edge contraction.
7. Kernighan, B. W. & Lin, S. (1970). An efficient heuristic procedure for partitioning graphs. Bell System Technical Journal 49(2).
8. Levinkov, E., Kirillov, A. & Andres, B. (2017). A comparative study of local search algorithms for correlation clustering. GCPR 2017.
9. Meilă, M. (2007). Comparing clusterings: an information based distance. Journal of Multivariate Analysis 98(5).

10. Papadakis, G., Skoutas, D., Thanos, E. & Palpanas, T. (2020). Blocking and filtering techniques for entity resolution: a survey. ACM Computing Surveys 53(2).
11. Christophides, V., Efthymiou, V., Palpanas, T., Papadakis, G. & Stefanidis, K. (2021). An overview of end-to-end entity resolution for big data. ACM Computing Surveys 53(6).
12. Hassanzadeh, O., Chiang, F., Lee, H. C. & Miller, R. J. (2009). Framework for evaluating clustering algorithms in duplicate detection. Proceedings of the VLDB Endowment 2(1).
13. Draisbach, U., Christen, P. & Naumann, F. (2019). Transforming pairwise duplicates to entity clusters for high-quality duplicate detection. ACM Journal of Data and Information Quality 12(1).
14. Christen, P. (2012). Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection. Springer.

Added, to be verified

15. Lange, J.-H., Karrenbauer, A. & Andres, B. (2018). Partial optimality and fast lower bounds for weighted correlation clustering. ICML 2018. Source of the persistency criterion of section 5.2 and of the cycle packing bound of section 5.4.
16. Alush, A. & Goldberger, J. (2012). Ensemble segmentation using efficient integer linear programming. IEEE Transactions on Pattern Analysis and Machine Intelligence 34(10). Decomposition of correlation clustering instances.
17. Gruenheid, A., Dong, X. L. & Srivastava, D. (2014). Incremental record linkage. Proceedings of the VLDB Endowment 7(9). Incremental clustering for record linkage, to be compared with section 9.
18. Saeedi, A., Peukert, E. & Rahm, E. (2017). Comparative evaluation of distributed clustering schemes for multi-source entity resolution. ADBIS 2017. Source of the MusicBrainz benchmarks.
19. Linacre, R., Lindsay, S., Manassis, T., Slade, Z. & Hepworth, T. (2022). Splink: free software for probabilistic record linkage at scale. International Journal of Population Data Science 7(3).
20. De Bruin, J. (2019). Python Record Linkage Toolkit. Zenodo. Distributes the Febrl data sets.